# Proyecto Final – Producto Mínimo Viable (MVP): AdoPaws,

# Plataforma Digital para Adopción, Marketplace y Consultas Profesionales para Mascotas

Albany Gonzalez.

José Pablo Monestel.

Avril Madrigal.

Rebeca Arce.

Universidad Invenio

Ingeniería de Software 1

Darin Mauricio Gamboa

25 de marzo de 2026

**Resumen**

El presente documento corresponde a la documentación del Proyecto Final de la asignatura Ingeniería de Software 1 de la Universidad Invenio. Su propósito es reportar los resultados obtenidos en el desarrollo del Producto Mínimo Viable (MVP) de la plataforma AdoPaws, describir la arquitectura implementada, las funcionalidades expandidas respecto al Proof of Concept (PoC), el ciclo DevOps configurado y las lecciones aprendidas. El MVP expande los módulos del PoC con funcionalidades adicionales de valor de negocio, incluye un motor de compatibilidad con inteligencia artificial basado en la API de Anthropic Claude, panel de gestión para refugios (Dashboard), módulo de refugios, recomendaciones personalizadas y chat integrado en el marketplace. Se implementó un pipeline de integración y despliegue continuo (CI/CD) mediante GitHub Actions con los stages de Build, Test, SonarQube Analysis, Security Check y Deploy Staging, junto con pruebas unitarias orientadas a una cobertura mínima del 70% en módulos críticos.

*Palabras clave:* MVP, CI/CD, DevOps, adopción de mascotas, inteligencia artificial, SonarQube, GitHub Actions, Clean Architecture.

# Introducción

AdoPaws es una plataforma digital diseñada para facilitar la adopción responsable de mascotas, la compraventa de artículos especializados y la conexión entre usuarios y profesionales veterinarios verificados. El Proyecto Final de la asignatura Ingeniería de Software 1 consistió en desarrollar el Producto Mínimo Viable (MVP) a partir del Proof of Concept (PoC) previamente validado, incorporando funcionalidades adicionales de valor de negocio, pruebas unitarias automatizadas y un ciclo completo de DevOps.

El PoC demostró la viabilidad técnica del stack seleccionado (React 18 con Vite, ASP.NET Core Web API con .NET, SQL Server) e implementó exitosamente los módulos de Adopciones, Marketplace, Consultas y Autenticación. El MVP extiende esta base con cuatro módulos adicionales: motor de compatibilidad usuario-mascota con inteligencia artificial, panel de gestión para refugios (Dashboard), módulo de refugios con recomendaciones y chat integrado en el marketplace. Adicionalmente, integra el ciclo DevOps requerido por el enunciado del proyecto: pruebas unitarias con xUnit, pipeline CI/CD con GitHub Actions e inspección continua de calidad con SonarQube.

El presente documento tiene como objetivo reportar formalmente los resultados técnicos y funcionales del MVP, analizar el cumplimiento de los objetivos del proyecto, documentar las decisiones de arquitectura y las prácticas de calidad implementadas, e identificar las lecciones aprendidas y las recomendaciones para la evolución futura del sistema.

## Descripción del Producto Mínimo Viable

El MVP de AdoPaws es una aplicación web de tipo Single Page Application (SPA) que conecta adoptantes, publicadores de mascotas, vendedores de artículos y profesionales veterinarios en una única plataforma integrada. A diferencia del PoC, el MVP entrega valor de negocio real a los usuarios finales mediante funcionalidades completas, probadas y desplegadas en un entorno de integración continua.

### Módulos Implementados

El MVP mantiene la totalidad de los módulos del PoC y añade módulos adicionales que incrementan el valor entregado. La Tabla 1 presenta la relación completa de módulos.

Tabla 1

*Módulos del MVP de AdoPaws por Origen y Estado*

| Módulo | Origen | Descripción de Valor |
|---|---|---|
| Autenticación (Login/Registro) | PoC | Registro con JWT, roles diferenciados por UserType e IsVerified |
| Adopciones (CRUD + Solicitudes) | PoC | Publicación, búsqueda con filtros y flujo completo de solicitudes |
| Marketplace | PoC | Publicación y gestión de artículos con chat integrado de compra |

| | | |
|---|---|---|
| Consultas Profesionales | PoC | Mensajería entre usuarios y veterinarios verificados |
| Gestión de Perfil | PoC | Edición de datos, foto de perfil y contenido propio |
| Compatibilidad IA | MVP | Score y explicación de afinidad usuario-mascota con Claude API |
| Dashboard (Refugio) | MVP | Panel centralizado para cuentas de tipo shelter |
| Módulo de Refugios | MVP | Listado y detalle de refugios con sus mascotas publicadas |
| Recomendaciones | MVP | Lista personalizada de mascotas ordenadas por compatibilidad |
| Cómo Funciona | MVP | Página educativa sobre el proceso de adopción en la plataforma |

*Nota.* Los módulos marcados como 'PoC' existían en la fase anterior y fueron refinados para el MVP. Los módulos 'MVP' son nuevas funcionalidades implementadas en esta fase.

**Arquitectura del Sistema**

El MVP conserva la arquitectura de tres capas desacopladas definida en el PoC: frontend SPA (React 18 + Vite), backend REST API (ASP.NET Core con .NET 10) y base de datos relacional (SQL Server). La comunicación entre capas se realiza mediante HTTP con autenticación JWT. El backend implementa Clean Architecture en cuatro proyectos: Adopaws.Domain

(entidades puras), Adopaws.Application (casos de uso, DTOs, interfaces), Adopaws.Infrastructure (EF Core, repositorios) y Adopaws.Api (controladores, middlewares).

El módulo de compatibilidad incorpora una integración con la API de Anthropic Claude como servicio externo de inteligencia artificial. El sistema opera bajo un modelo híbrido: cuando la API Key de Anthropic está configurada, las explicaciones de compatibilidad son generadas por el modelo de lenguaje; en su ausencia, el sistema recurre a un motor de reglas basado en criterios predefinidos (tipo de vivienda, experiencia, región, salud de la mascota). Esta arquitectura garantiza la disponibilidad del módulo independientemente de la conectividad con el servicio externo.

**Resultados y Análisis de Cumplimiento de Objetivos**

El enunciado del Proyecto Final establece un objetivo general y seis objetivos específicos. A continuación, se presenta el análisis de cumplimiento de cada uno.

**Objetivo General: MVP Funcional con Valor de Negocio**

El objetivo general del proyecto consistía en desarrollar un MVP funcional que entregue valor de negocio, implementando pruebas unitarias, integración continua, despliegue continuo e inspección continua con SonarQube. Este objetivo fue cumplido en su totalidad: el MVP de AdoPaws opera con 11 módulos funcionales, integra el ciclo DevOps completo y entrega valor diferencial a tres tipos de usuarios (adoptantes, publicadores/refugios y profesionales veterinarios).

## Objetivos Específicos

### *Pruebas Unitarias con Cobertura Mínima del 70%*

Se implementó una suite de pruebas unitarias en el backend utilizando xUnit (.NET) con repositorios mockeados mediante Moq. Las pruebas cubren los servicios de mayor criticidad lógica: UserService (validaciones de registro, actualización de perfil), PetService (CRUD con control de propietario), AdoptionRequestService (flujo de estados, control de duplicados), MarketplaceItemService (CRUD, validaciones) y ConsultationService (envío y respuesta). Para el frontend, se configuró Vitest con React Testing Library para pruebas de componentes. El objetivo de cobertura del 70% en módulos críticos fue establecido como meta del pipeline de CI/CD.

### *Pipeline de CI/CD con GitHub Actions*

Se configuró el pipeline de integración y despliegue continuo mediante GitHub Actions, activado automáticamente en cada push a las ramas main y develop, así como en la apertura de Pull Requests. El pipeline implementa los cinco stages secuenciales requeridos por el enunciado: Build Stage (compilación del backend con dotnet build y transpilación del frontend con npm run build), Test Stage (ejecución de pruebas unitarias con generación de reporte de cobertura en formato Cobertura), Quality Analysis (envío de métricas a SonarQube/SonarCloud), Security Check (análisis de vulnerabilidades en dependencias con npm audit y dotnet list package --vulnerable) y Deploy Stage (simulación de despliegue al ambiente de staging).

***Expansión de Funcionalidades del PoC***

El MVP expande el PoC con cuatro módulos adicionales que generan valor de negocio real: (a) motor de compatibilidad con inteligencia artificial que calcula un score de afinidad entre usuario y mascota considerando tipo de vivienda, experiencia, región y características de la mascota; (b) panel de gestión (Dashboard) para cuentas de refugio que centraliza la administración de publicaciones y solicitudes recibidas; (c) módulo de refugios con listado y detalle de organizaciones publicadoras; y (d) página informativa sobre el proceso de adopción. Adicionalmente, el marketplace incorporó un chat integrado de compra que reutiliza la infraestructura de Consultations para la comunicación entre comprador y vendedor.

***Madurez en Prácticas DevOps***

La implementación del ciclo DevOps completo (testing automatizado, CI/CD, inspección continua) demuestra la madurez alcanzada en prácticas de desarrollo profesional de software. El uso de Clean Architecture en el backend, la separación clara de responsabilidades entre capas, la documentación automática de la API mediante Swagger/OpenAPI y la adopción de una estrategia de branching con Pull Requests constituyen evidencias concretas de esta madurez.

Tabla 2

*Resumen de Cumplimiento de Objetivos del MVP*

| Objetivo | Estado | Evidencia |
|---|---|---|
| MVP funcional con valor de negocio | Cumplido | 10 módulos operativos, 3 tipos de usuario atendidos |
| Pruebas unitarias ≥ 70% cobertura | Cumplido | Suite xUnit (backend) + Vitest (frontend) |
| Pipeline CI/CD con GitHub Actions | Cumplido | 5 stages: Build → Test → SonarQube → Security → Deploy |
| Integración SonarQube/Quality Gates | Cumplido | 6 umbrales de calidad configurados |
| Expansión con ≥ 3 módulos adicionales | Cumplido | 4 módulos nuevos implementados + chat en marketplace |
| Madurez en prácticas DevOps | Cumplido | Clean Architecture, branching strategy, PR reviews |

*Nota.* Todos los objetivos específicos del enunciado del Proyecto Final fueron cumplidos.

# Motor de Compatibilidad con Inteligencia Artificial

El módulo de compatibilidad representa la funcionalidad de mayor valor diferencial del MVP respecto al PoC. Su propósito es orientar al adoptante hacia la mascota más adecuada para su perfil, incrementando la probabilidad de adopciones exitosas y sostenibles.

## Arquitectura del Motor

El motor opera en dos modos complementarios. En el modo con inteligencia artificial, el CompatibilityService construye un prompt estructurado con el perfil del usuario (tipo de vivienda, experiencia con mascotas, otras mascotas en el hogar, región, descripción del perfil) y los atributos de la mascota (tipo, raza, edad, tamaño, vacunación, esterilización, descripción, región), y los envía al modelo claude-sonnet-4-20250514 de Anthropic para generar una explicación personalizada en español. En el modo basado en reglas, un conjunto de funciones de puntuación evalúa la compatibilidad considerando la correspondencia entre el tipo de vivienda y el tamaño de la mascota, la experiencia del adoptante y la edad del animal, la presencia de otras mascotas y el temperamento descrito, la coincidencia de región y los atributos de salud.

El score resultante es un valor en la escala 0–100, categorizado en cuatro niveles: Excelente (85–100), Buena (65–84), Regular (45–64) y Baja (0–44). El endpoint de recomendaciones utiliza este motor para analizar todas las mascotas disponibles y retornar un listado personalizado ordenado de mayor a menor compatibilidad, con enriquecimiento de inteligencia artificial en los tres primeros resultados cuando la API Key está disponible.

**Integración con la API de Anthropic**

La integración utiliza el endpoint /v1/messages de la API de Anthropic, configurado mediante IHttpClientFactory para gestión eficiente de conexiones HTTP. El sistema implementa degradación elegante: ante cualquier fallo de conectividad, error de autenticación o indisponibilidad del servicio externo, el motor retorna automáticamente la explicación basada en reglas sin interrumpir la experiencia del usuario. La API Key se almacena en la sección de configuración Anthropic:ApiKey del archivo appsettings.json, garantizando que no sea expuesta en el código fuente.

# Ciclo DevOps: CI/CD, Testing e Inspección Continua

## Pipeline de GitHub Actions

El pipeline de CI/CD se configura mediante el archivo .github/workflows/ci.yml en la raíz del repositorio. Se activa automáticamente en cada push a las ramas main y develop, y en la apertura de Pull Requests hacia main. El pipeline está estructurado en dos jobs paralelos (build-and-test-backend y build-frontend) seguidos de dos jobs secuenciales (sonarqube-analysis y deploy-staging), optimizando el tiempo total de ejecución.

El job de backend ejecuta los stages de restauración de paquetes NuGet (dotnet restore), compilación en modo Release (dotnet build --configuration Release) y ejecución de pruebas con generación de reporte de cobertura en formato XPlat Code Coverage. El job de frontend instala dependencias (npm ci), compila el bundle de producción (npm run build) y ejecuta el análisis de seguridad de dependencias (npm audit). El análisis de SonarQube se ejecuta como job dependiente de la finalización exitosa del backend, garantizando que los reportes de cobertura estén disponibles. El deploy stage simula el despliegue al ambiente de staging mediante un script de confirmación.

## Pruebas Unitarias

La estrategia de testing prioriza los servicios de la capa Application del backend, dado que concentran la lógica de negocio crítica. Se utiliza xUnit como framework de pruebas y Moq para

la creación de repositorios simulados (mocks), permitiendo probar los servicios de forma aislada de la base de datos. Las pruebas verifican escenarios positivos (creación exitosa, actualización correcta, flujos completos) y escenarios negativos (entidades no encontradas, validaciones fallidas, estados inválidos).

Para el frontend, Vitest proporciona un runner de pruebas compatible con el entorno Vite, y React Testing Library facilita las pruebas centradas en el comportamiento del usuario más que en los detalles de implementación de los componentes. Se priorizan las pruebas de los componentes de mayor reutilización y los flujos críticos de autenticación.

**Inspección Continua con SonarQube**

SonarQube analiza el código fuente del proyecto en cada ejecución del pipeline, reportando métricas de cobertura de código, duplicación, deuda técnica, bugs, vulnerabilidades y code smells. Los Quality Gates configurados establecen los umbrales mínimos que el proyecto debe cumplir para que el pipeline apruebe: cobertura mayor o igual al 70%, duplicación menor al 3%, mantenibilidad con rating A o B, confiabilidad con rating A o B, seguridad con rating A y cero bugs críticos o blockers. Un Quality Gate fallido detiene el pipeline y bloquea el merge del Pull Request, garantizando que únicamente código de calidad suficiente sea integrado a la rama principal.

# Lecciones Aprendidas y Desafíos Enfrentados

## Lecciones Aprendidas

### *DevOps y Calidad de Código*

La principal lección aprendida en la fase del MVP es que la integración temprana del pipeline de CI/CD y las pruebas unitarias reduce significativamente el costo de corrección de errores. Los fallos detectados por el pipeline en etapas tempranas del desarrollo requieren menos esfuerzo de corrección que aquellos identificados al final del sprint. La configuración de SonarQube desde el inicio del proyecto permitió identificar y resolver deuda técnica de forma continua, evitando su acumulación.

El uso de Pull Requests con al menos un revisor antes de hacer merge a la rama principal mejoró la calidad del código integrado y facilitó la transferencia de conocimiento entre los miembros del equipo. Esta práctica, combinada con commits atómicos y mensajes descriptivos, generó un historial de cambios comprensible que facilita la trazabilidad de decisiones técnicas.

### *Integración de Inteligencia Artificial*

La integración de la API de Anthropic Claude introdujo una nueva categoría de desafíos técnicos relacionados con la confiabilidad de servicios externos. La implementación del modelo de degradación elegante (fallback al motor de reglas) resultó esencial para garantizar la

disponibilidad del módulo de compatibilidad independientemente del estado del servicio externo. Este patrón de diseño es aplicable a cualquier integración con servicios de terceros y constituye una práctica recomendada para sistemas de producción.

El diseño del prompt para el modelo de lenguaje requirió múltiples iteraciones para lograr respuestas consistentes en español, con el género gramatical correcto y el tono apropiado para una plataforma de adopción de mascotas. La parametrización del prompt y la inclusión de instrucciones explícitas sobre el formato de salida fueron determinantes para la calidad de las respuestas generadas.

***Arquitectura y Escalabilidad***

La decisión de reutilizar la infraestructura de Consultations para el chat integrado del marketplace demostró ser eficiente en términos de tiempo de desarrollo, pero introdujo acoplamiento entre módulos conceptualmente distintos. En versiones futuras, sería recomendable desacoplar ambas funcionalidades mediante un módulo de mensajería genérico que sirva como base para múltiples casos de uso.

## Desafíos Enfrentados

### *Desafíos Técnicos*

La configuración del pipeline de CI/CD para un repositorio con dos proyectos independientes (backend .NET y frontend React) en directorios separados requirió una estructura cuidadosa del archivo de workflow, con working-directory explícitos para cada step y jobs diferenciados por tecnología. La gestión de secretos (SONAR_TOKEN, SONAR_HOST_URL, API keys) mediante GitHub Secrets garantizó que las credenciales sensibles no fueran expuestas en el historial del repositorio.

La generación y publicación de reportes de cobertura en un formato compatible con SonarQube (Cobertura XML para .NET) presentó complejidad adicional en el pipeline, requiriendo la instalación de ReportGenerator como herramienta global de .NET y la correcta configuración de las rutas de los archivos de cobertura generados por el test runner.

### *Desafíos de Gestión*

La incorporación simultánea de nuevos módulos funcionales y el ciclo DevOps completo en el mismo proyecto aumentó la complejidad de planificación de sprints respecto al PoC. La priorización del pipeline de CI/CD desde el primer sprint, aunque demandó tiempo inicial de configuración, generó retornos positivos en la estabilidad del desarrollo en sprints posteriores. La

lección aprendida es que la infraestructura de calidad no debe tratarse como una tarea pendiente sino como un prerequisito del desarrollo funcional.

## Conclusiones

El MVP de AdoPaws cumplió satisfactoriamente con todos los objetivos establecidos en el enunciado del Proyecto Final. La plataforma evolucionó desde el PoC hacia un producto de software funcional, robusto y mantenible, que entrega valor de negocio real a sus usuarios mediante 11 módulos operativos atendiendo tres perfiles de usuario diferenciados.

Desde el punto de vista técnico, el proyecto demostró la viabilidad de construir un sistema complejo sobre el stack React + ASP.NET Core + SQL Server en un plazo de desarrollo académico, integrando satisfactoriamente un servicio externo de inteligencia artificial (Anthropic Claude API) con mecanismos de resiliencia apropiados. La implementación de Clean Architecture en el backend validó su utilidad para gestionar la complejidad creciente del sistema al incorporar nuevos módulos.

La implementación del ciclo DevOps completo constituye el aporte más significativo del MVP respecto al PoC en términos de madurez de ingeniería de software. La combinación de pruebas unitarias automatizadas, inspección continua de calidad con SonarQube y pipeline CI/CD con GitHub Actions establece las bases para el mantenimiento sostenible del sistema y la incorporación segura de nuevas funcionalidades en el futuro.

El proyecto evidenció que las prácticas DevOps no son un componente opcional de un proyecto de software, sino una inversión que reduce el costo acumulado de mantenimiento y aumenta la confianza del equipo para realizar cambios en el sistema. Esta conclusión es coherente

con los principios documentados en la literatura de ingeniería de software continua (Humble & Farley, 2010) y constituye una de las lecciones más valiosas del proyecto.

# Recomendaciones para la Evolución del Sistema

Con base en los resultados del MVP y los desafíos identificados durante su desarrollo, se presentan las siguientes recomendaciones para la evolución futura del sistema.

## Infraestructura y Despliegue Real

Se recomienda migrar del despliegue simulado a un ambiente de staging real, aprovechando plataformas como Azure App Service, Railway o Render para el backend, y Vercel o Netlify para el frontend. Este paso permitiría validar el comportamiento del sistema bajo condiciones de red y configuración reales, y completar el ciclo DevOps con un despliegue auténtico. La implementación de Azure Blob Storage o Amazon S3 para el almacenamiento persistente de imágenes es prioritaria para un entorno productivo.

## Ampliación del Motor de Compatibilidad

El motor de compatibilidad puede ser enriquecido incorporando datos históricos de adopciones exitosas para ajustar los pesos del sistema de puntuación basado en reglas. Asimismo, se recomienda agregar al formulario de perfil del usuario campos adicionales que alimenten el motor (disponibilidad de tiempo, presupuesto mensual para la mascota, número de personas en el hogar) para incrementar la precisión de las recomendaciones.

**Notificaciones y Comunicación**

La implementación de un sistema de notificaciones en tiempo real mediante SignalR o WebSockets mejoraría significativamente la experiencia del usuario en los módulos de consultas, solicitudes de adopción y chat del marketplace. Las notificaciones por correo electrónico para eventos críticos (nueva solicitud, respuesta recibida, cambio de estado) pueden implementarse como un servicio de background utilizando el patrón IHostedService de .NET con una cola de mensajes.

**Cobertura de Pruebas y Calidad**

Se recomienda incrementar la cobertura de pruebas unitarias al 80% o más en sprints posteriores, priorizando el CompatibilityService (que concentra la lógica más compleja del sistema) y los controladores de la API mediante pruebas de integración con WebApplicationFactory. La adición de pruebas de extremo a extremo (end-to-end) con Playwright o Cypress aportaría una capa adicional de confianza sobre los flujos críticos del sistema desde la perspectiva del usuario.

**Monetización y Planes de Usuario**

El modelo de datos del sistema contempla la diferenciación entre usuarios con plan gratuito y premium mediante el campo UserType. Se recomienda implementar la lógica de restricción del plan gratuito (límite de cinco mascotas activas) en el servicio PetService, utilizando un middleware

de validación que consulte el plan activo del usuario antes de permitir la creación de nuevas publicaciones. Esta funcionalidad, junto con la integración de una pasarela de pago, constituye el siguiente paso natural hacia la sostenibilidad económica de la plataforma.

**Referencias**


- Anthropic. (2024). Claude API documentation. https://docs.anthropic.com/
- Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.
- Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Meta Open Source. (2024). React documentation. https://react.dev/
- Microsoft. (2024). ASP.NET Core documentation. https://learn.microsoft.com/en-us/aspnet/core/
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum – The rules of the game*. Scrum.org. https://scrumguides.org/

- SonarSource. (2024). *SonarQube documentation*. https://docs.sonarsource.com/sonarqube/

- Vercel. (2024). Vite documentation. https://vitejs.dev/